Cache-Conscious Design for Real-Time Molecular Simulation on Laptops

CSE340: Research-Based Assessment (Question 18)

Student Name: Muaz Ibne Ashraf | Student ID: 21101177

Introduction

A bioinformatical educational application runs a real-time molecular dynamics (MD) simulation on the student laptops (standard specifications: Intel Core i5, four cores, 3 MB L3 cache, 8 GB RAM). It simulates atomic coordinates at about 2ms/step, adds three-dimensional information about molecular structures, and can record frames to be later replayed. Using simulations of closer to one thousand atoms, the students note performance reductions, jitter in the user interface, and unstable CPU usage, even with the use of OpenMP parallelization. In this report, it is argued that the observed limits are primarily due to cache capacity and contention, and not to floating-point performance. Thus, it suggests a redesign that is cache aware to ensure interactivity in commodity laptops.

Background / Related Work

The current central processing units rely on the hierarchy of the multi-level caches (L1, L2, L3) to hide the latency of the DRAM. Algorithms can be very performance-intensive when they utilize locality of timing to reuse a data before it is evicted and locality of space, receiving adjacent memory addresses. Blocking or tiling strategies restructure loop nests so as to run on the sub-problems that fit within a known cache, reducing capacity- and conflict-misses and often achieving speedups in the range of a dozen or more times in kernel-like matrix operations.[1]. Shared-memory parallelism, such as OpenMP, can scale efficiently only when outer loops are shared among threads and the inner kernels respect locality constraints (that is,Upper structures-OFRA layouts,blocked neighbor loops, and prefetching)[2]. The presence of threads does not entirely fix bandwidth bottlenecks, or cache thrashing; scalable performance is only achieved when every thread working set has been referenced into the appropriate level of cache.[3].

Methodology / System Design

A. Where cache contention arises

The application performs: (i) force accumulation over neighbor tiles (matrix-like), (ii) position/velocity updates, (iii) 3D rendering, and (iv) optional frame export. These tasks *share the 3 MB L3* across four cores; the renderer and recorder introduce streaming reads/writes that evict hot simulation lines.

Working-set model (given constraints). Each atom needs 120 B per frame. Matrix-heavy operations touch 20% of atoms per update; hence the per-step touched data are

$$W(N) = 0.2N \times 120\ \text{B} = 24N\ \text{B}.$$

If the *entire* frame must fit in L3: $N \leq \lfloor 3{,}145{,}728/120 \rfloor \approx 26{,}214$ atoms. If only the *touched* set must fit a cache of size C, then $24N \leq C$.

Cache available to a thread	C **(bytes)**	**Max** N **from** $24N \leq C$	**Notes**
~L2 per-core (256 KiB)	262,144	10,922	Target for tight blocking
Share of L3 (3 MB / 4 threads)	786,432	32,768	Fair-share assumption
Entire L3 (3 MB)	3,145,728	131,072	Upper bound, optimistic

Table 1: Per-thread cache budget versus atoms touched each step (20% of atoms).

B. Architecture-level changes

1) Loop tiling / cache blocking over spatial cells. Partition atoms into *cell-blocks* (e.g., 2–8k atoms ⇒ ~240–960 KB at 120 B/atom) so each OpenMP thread reuses a small in-cache working set: load positions/velocities → iterate neighbor tiles → accumulate forces → integrate → render glyphs of that block, then advance. This mirrors blocking in DGEMM and drastically reduces capacity/conflict misses [1].

2) Structure-of-Arrays (SoA). Store coordinates, velocities, and types in separate arrays: `x[], y[], z[], vx[], vy[],` SoA maximizes contiguous loads/stores and SIMD efficiency; AoS induces striding.

3) Software prefetching. Insert prefetches for predictable neighbor streams (e.g., `__builtin_prefetch(ptr,0,3)`) a few iterations ahead; keep the distance tunable per platform.

4) False sharing avoidance. Pad per-thread accumulators to a cache line and use OpenMP `reduction` only at block boundaries to limit coherence traffic.

5) Simulation–UI/Export decoupling (pipeline). Use a lock-free SPSC ring buffer between simulation and consumers:

`SIM step threads` ⇒ `snapshot queue` ⇒ {`renderer`, `encoder`}.

Pin renderer/encoder to separate cores. Use non-temporal/streaming writes for export to reduce L3 pollution by write-only data.

6) Scheduling and chunking. Parallelize the outer block loop:

```
#pragma omp parallel for schedule(static, BLOCKS_PER_CHUNK)
for (int b = 0; b < nBlocks; ++b) {
  prefetch(block[b+PREFDIST]);
  compute_forces_block(b);
  integrate_block(b);
  update_renderer_vbo(b);
  if (recording) enqueue_snapshot(b);
}
```

Discussion

Impact: The blocking scheme gathers far-between accesses in contiguous bursts and in this fashion lists an L2- and L3-working set of the size of that block in the working set. The method minimizes capacity and conflict misses and often gives speedups many times higher than the inner loops with matrices[1]. With 2–8k atom tiles (240–960 KB), it does not leave its shared L2/L3 cache and can thus stabilize CPU usage and reduced frame-to-frame jitter. The pipeline structure prevents recorder eviction of popular simulation lines; software prefetch alleviates delay during movement into the tile adjacent in the grid. This decrease in

off-book traffic would in turn be a decrease in the amount of energy used to create each frame and this would produce laptops that run cooler and quieter.

Trade-offs / limitations: The size of the tile depends on hardware characteristics, which include cache size, associativity and prefetching, hence tuning is needed on a platform by platform basis. Blocking raises code complexity, and in careless application can slow down a deficient vectorization, which can be addressed with adoption of a Structure of Arrays representation. A congested example involving too much compression may turn the encoder into the system bottleneck; the total pipeline will bring this concern under control but does not smoothly lead to quicker input /output throughput. Amdahl law remains a constraint on the attainable speedup, due to serial user-interface and synchronization paths.[2].

Sustainability and societal impact: Caching to waste reduction optimises the DRAM reuse traffic and is the most energy consumed part of the execution, allowing enhancement of DRAM reuse of the existing laptops in order to achieve more intensive simulation using the already available hardware. Better data locality is feasible, increasing device lifetime, reducing laboratory energy consumption, and ultimately broadening access to high-field visualization of molecular dynamics in a classroom and other low-resource contexts.[3].

Conclusion

The identified slows and jitter can be mainly attributed to cache-capacity contention and bandwidth interference between simulation, rendering and recording of the 4-core 3MB L3 laptops. An analysis of the per-step working set as $24N$ bytes indicates why naive scattered iterations deteriorate the cache performance. A pragmatic redesign, which includes a Structur of Arrays (SoA) layout, spatial blocking, OpenMP, with statical chunk and padded reduction, software prefetching and a decoupled UI/export pipeline with stream writes, met side-by-side to the principles of a memory-hierarchy and once again offers smooth, real-a time interactivity at significantly higher atom count.

References

[1] Patterson, D. A., & Hennessy, J. L. (2021). *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface* (2nd ed.).

[2] OpenMP Architecture Review Board. (2023). *OpenMP Application Programming Interface* (Version 5.2). `https://www.openmp.org/specifications/`

[3] McCalpin, J. D. (1995). STREAM: *Sustainable Memory Bandwidth in High Performance Computers* (Technical Report). `https://www.cs.virginia.edu/stream/`